Anatomy of an Android Application

UNIT 1 : PART B

Android Application Components

1. Activities
2. Services
3. Broadcast Receivers
4. Content Providers

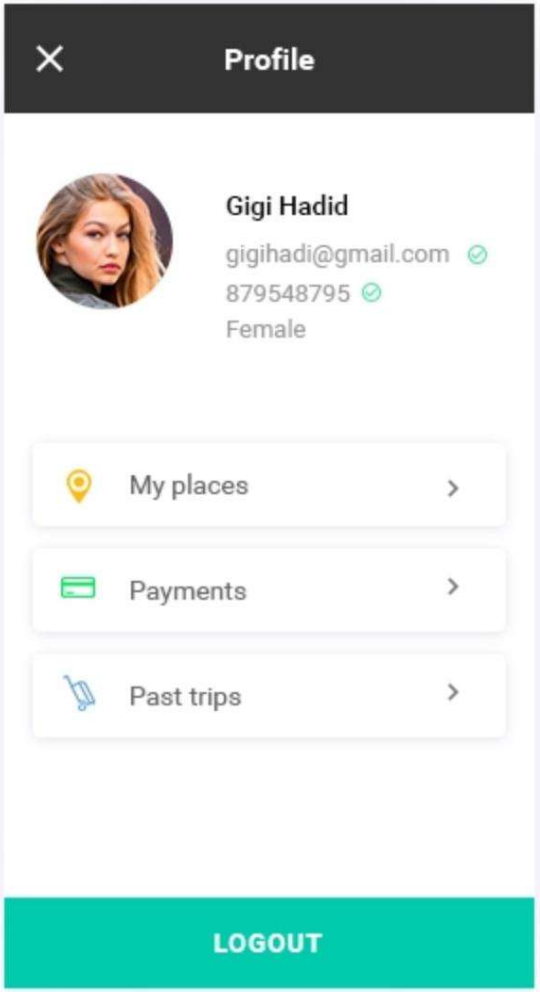
1. Activity

An activity provides the window in which the app draws its UI. This window typically fills the screen, but may be smaller than the screen and float on top of other windows. Generally, one activity implements one screen in an app.

Most apps contain multiple screens, which means they comprise multiple activities. Typically, one activity in an app is specified as the *main activity*, which is the first screen to appear when the user launches the app.

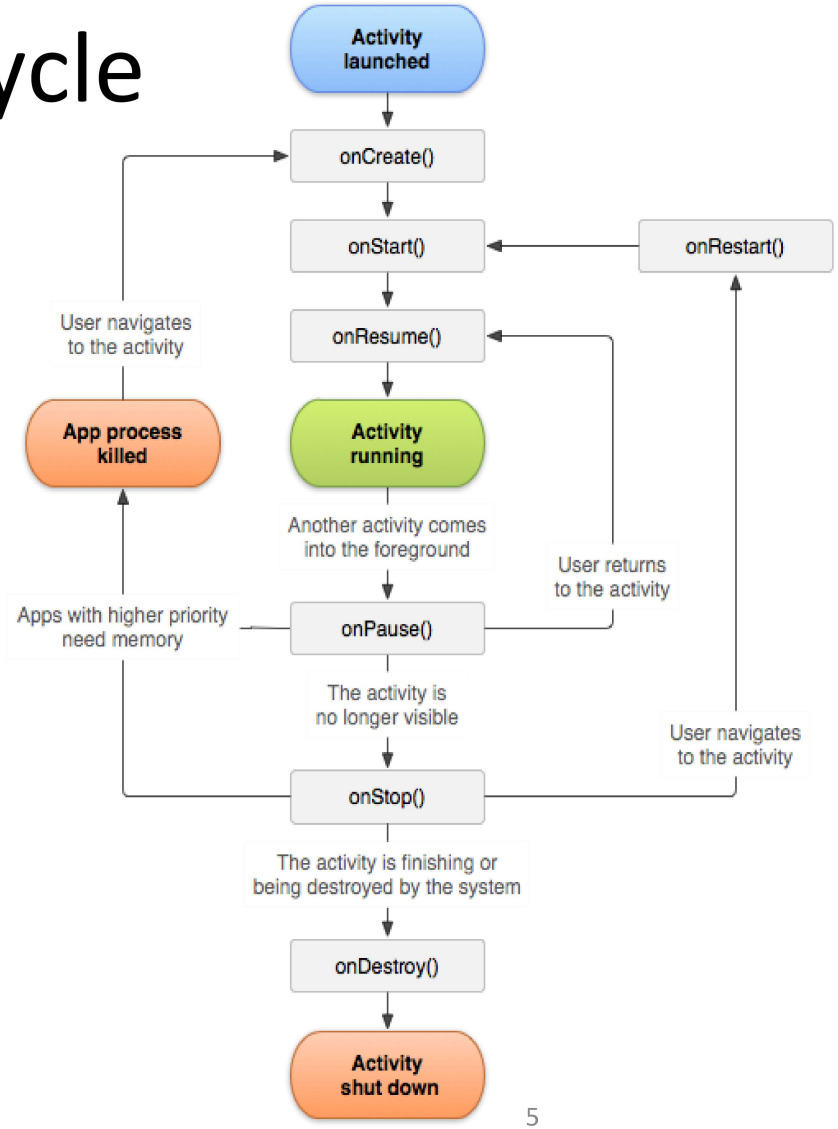
Activity

Unlike programming paradigms in which apps are launched with a main() method, the Android system initiates code in an Activity instance by invoking specific callback methods that correspond to specific stages of its lifecycle.



Activity Lifecycle

1. onCreate()
2. onStart()
3. onResume()
4. onPause()
5. onStop()
6. onDestroy()



Activity Lifecycle

To navigate transitions between stages of the activity lifecycle, the Activity class provides a core set of six callbacks:

onCreate(), onStart(), onResume(), onPause(), onStop(), and onDestroy().

The system invokes each of these callbacks as the activity enters a new state.

Activity Lifecycle

1. onCreate(): Called When: The activity is first created.

2. onStart(): Called When: Activity becomes visible to the user. Prepare UI elements for user interaction. Start animations, listeners, or sensor listeners.

3. onResume(): Called When: Activity comes to the foreground and is interactive. Resume updates like live data or animations. Start camera, audio, or sensor-based operations.

Activity Lifecycle

4. onPause(): Called When: Activity goes into the background (e.g., another activity appears). Pause ongoing processes like video playback. Save temporary states like scroll positions.

5. onStop(): Called When: Activity is no longer visible (e.g., user switches to another app). Release resources like network connections or Stop background tasks to free memory.

6. onRestart(): Called When: Activity is restarted after being stopped. Reinitialize resources before resuming. Restore UI changes if necessary.

Activity Lifecycle

7. onDestroy(): Called When: Activity is being destroyed (e.g., user exits or system kills it). Clean up resources to prevent memory leaks. Save persistent data if necessary.

Activity Lifecycle

Practical Use Cases

onCreate() : Initialize the app's UI.

onStart() : Start background music.

onResume() : Resume video playback.

onPause() : Pause video or save draft data.

onStop() : Release memory-heavy tasks.

onDestroy() : Close database connections.

Activity Lifecycle

1. onCreate() — The "Setup"

This is the birth of the activity. It happens only **once** when the app is first launched.

What happens: The layout is created (setContent), and variables are initialized.

Camera Example: You tap the Camera icon. The app starts "warming up" and preparing the buttons.

Status: Not visible yet.

2. onStart() — The "Appearance"

The app is now visible on the screen, but you can't interact with it yet.

What happens: The system prepares the app to come to the foreground.

Camera Example: The camera screen flashes onto your display.

Status: Visible, but not "active."

Activity Lifecycle

3. onResume() — The "Interaction"

This is the stage where the app is fully running and the user is interacting with it.

What happens: The app stays in this state as long as it has the user's focus.

Camera Example: The "Viewfinder" starts moving. You can now see through the lens and tap the "Capture" button.

Status: Running & Interactive.

4. onPause() — The "Interruption"

The app is still partly visible, but something else has taken the focus (like a pop-up or a split-screen).

What happens: You should stop doing things that consume battery (like heavy animations).

Camera Example: A "Low Battery" warning pops up over your camera. The camera is still visible behind the pop-up, but you can't take a photo right now.

Status: Partially hidden.

Activity Lifecycle

5. onStop() — The "Hidden"

The app is no longer visible at all. The user has pressed the "Home" button or switched to a different app.

What happens: The app saves data and stops using heavy resources.

Camera Example: You switch from the Camera to Instagram. The Camera app is "frozen" in the background.

Status: Hidden.

6. onDestroy() — The "Death"

The activity is being completely removed from the phone's memory.

What happens: This happens if you swipe the app away from the "Recent Apps" list or if the system needs more RAM.

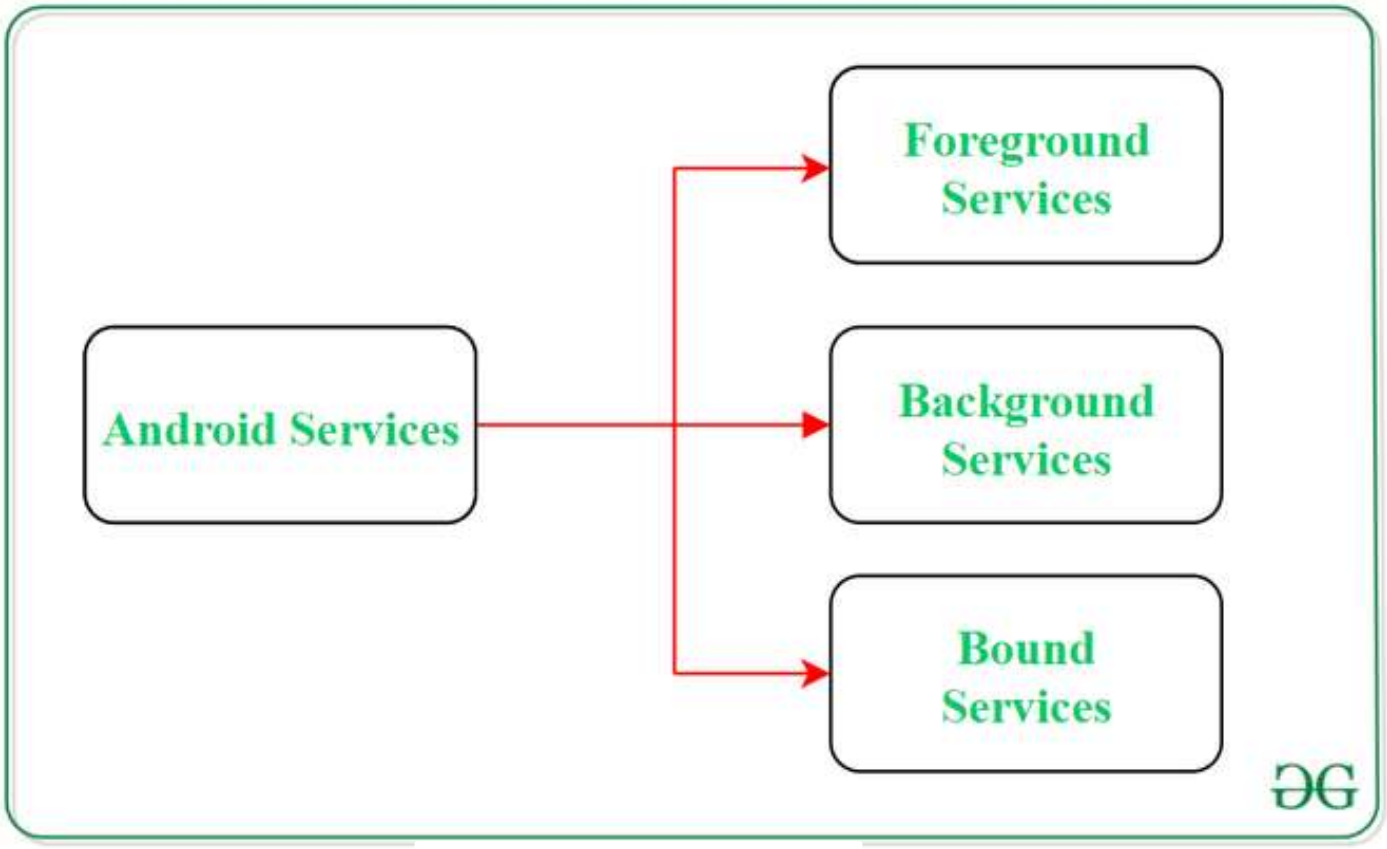
Camera Example: You close the Camera app completely to save battery. All camera resources are released.

Status: Gone.

2. Services

- Services in Android are a special component that facilitates an application to run in the background in order to perform long-running operation tasks.
- The prime aim of a service is to ensure that the application remains active in the background so that the user can operate multiple applications at the same time.
- A user-interface is not desirable for android services as it is designed to operate long-running processes without any user intervention.
- A service can run continuously in the background even if the application is closed or the user switches to another application.

Types of Adroid Services



1. Foreground Services:

Services that notify the user about its ongoing operations are termed as Foreground Services. Users can interact with the service by the notifications provided about the ongoing task. Such as in downloading a file, the user can keep track of the progress in downloading and can also pause and resume the process.

2. Background Services:

Background services do not require any user intervention. These services do not notify the user about ongoing background tasks and users also cannot access them. The process like schedule syncing of data or storing of data fall under this service.

3. Bound Services:

This type of android service allows the components of the application like activity to bound themselves with it. Bound services perform their task as long as any application component is bound to it. More than one component is allowed to bind themselves with a service at a time. In order to bind an application component with a service **bindService()** method is used.

3. Content Providers

It is used to manage and persist the application data also typically interacts with the SQL database. They are also responsible for sharing the data beyond the application boundaries.

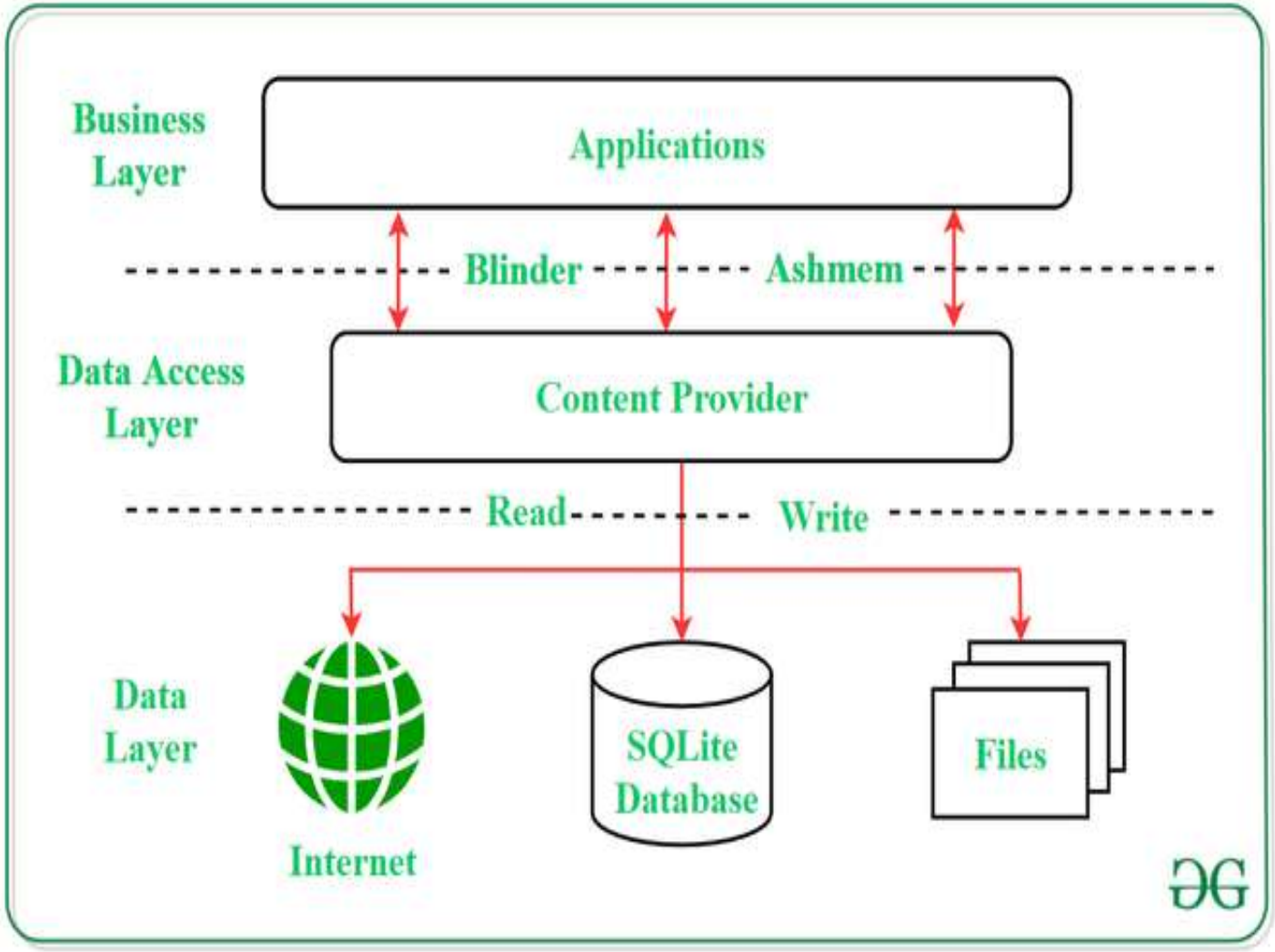
The Content Providers of a particular application can be configured to allow access from other applications, and the Content Providers exposed by other applications can also be configured.

In Android , Content Providers are a very important component that serves the purpose of a relational database to store the data of applications.

The role of the content provider in the android system is like a central repository in which data of the applications are stored, and it facilitates other applications to securely access and modifies that data based on the user requirements.

Android system allows the content provider to store the application data in several ways. Users can manage to store the application data like images, audio, videos, and personal contact information by storing them in **SQLite Database , in files** , **or even on a network** .

In order to share the data, content providers have certain permissions that are used to grant or restrict the rights to other applications to interfere with the data.


Business Layer
Applications
Blinder
Ashmem
Data Access Layer
Content Provider
Read
Write
Data Layer
Internet
SQLite Database
Files

Operations in Content Provider

Four fundamental operations are possible in Content Provider namely **Create** , **Read** , **Update** , and **Delete** . These operations are often termed as **CRUD operations** .

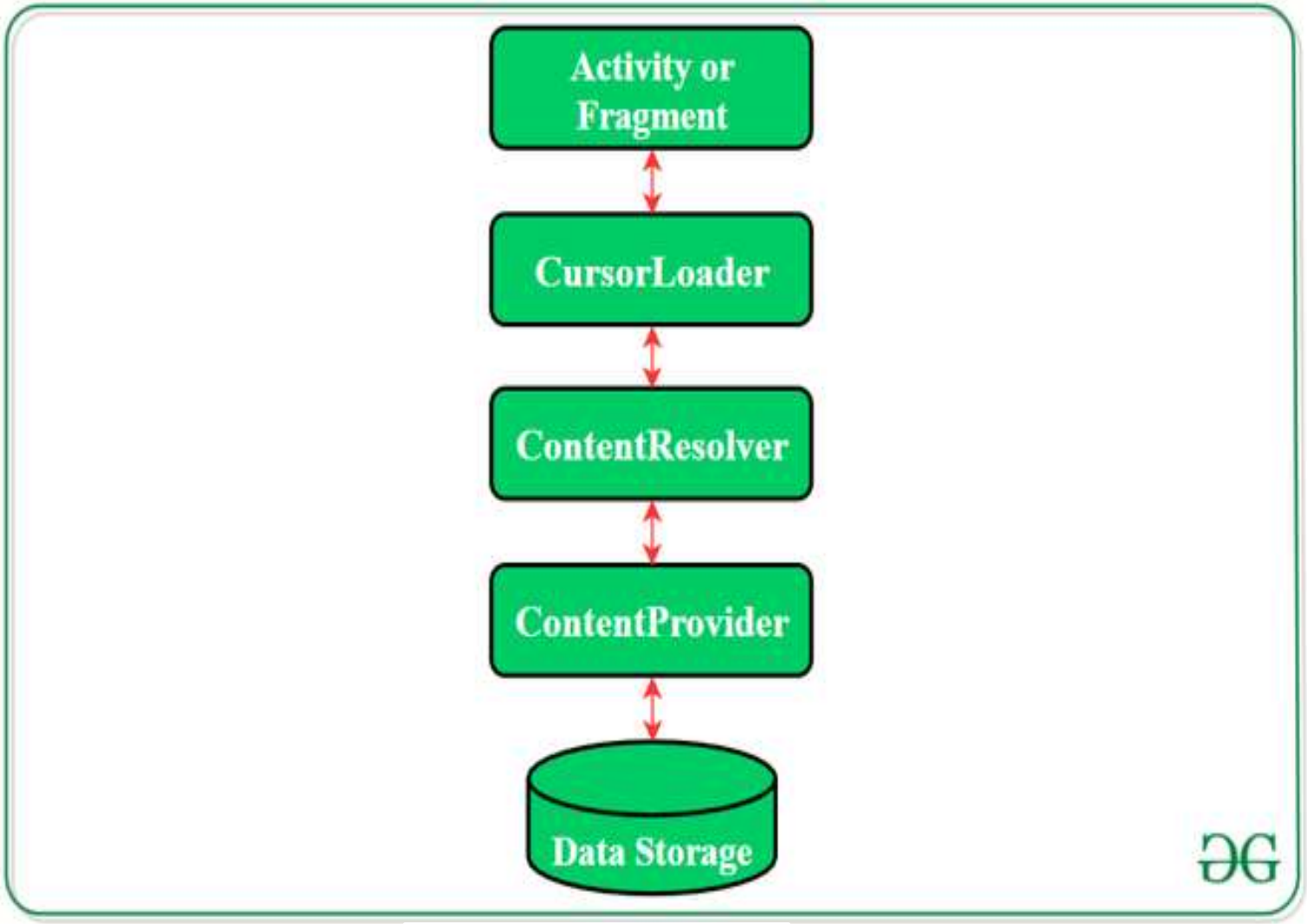
- **Create:** Operation to create data in a content provider.
- **Read:** Used to fetch data from a content provider.
- **Update:** To modify existing data.
- **Delete:** To remove existing data from the storage.

Working of the Content Provider

UI components of android applications like Activity and Fragments use an object **CursorLoader** to send query requests to **ContentResolver.**

The ContentResolver object sends requests (like create, read, update, and delete) to the **ContentProvider** as a client.

After receiving a request, ContentProvider process it and returns the desired result. Below is a diagram to represent these processes in pictorial form.


Activity or Fragment
CursorLoader
ContentResolver
ContentProvider
Data Storage

4. Broadcast Receiver

Broadcast in android is the system-wide events that can occur when the device starts, when a message is received on the device or when incoming calls are received, or when a device goes to airplane mode, etc. Broadcast Receivers are used to respond to these system-wide events. Broadcast Receivers allow us to register for the system and application events, and when that event happens, then the register receivers get notified. There are mainly two types of Broadcast Receivers:

- **Static Broadcast Receivers:** These types of Receivers are declared in the manifest file and works even if the app is closed.
- **Dynamic Broadcast Receivers:** These types of receivers work only if the app is active or minimized.